

Самокоррекция (ECC - Error Correcting Code)

Во времена, когда электроника была ненадёжной, способность обнаруживать (или даже исправлять) ошибки в памяти была бесценной. Причем делалось это за счет некоторой избыточности, не затрагивая физику процесса чтения/записи. С тех пор надёжность хранения памяти существенно выросла, но в ответственных задачах и/или в агрессивных условиях радиации, когда вероятность ошибки достаточно велика, память с коррекцией ошибок по-прежнему применяют. Как же это работает?

Просто обнаружить ошибку в блоке данных на первый взгляд не очень сложная задача, например, введём дополнительный разряд - контроль четности. Работает он так - подсчитываем при записи бинарного блока число единиц в его разрядах, если оно четное, результат равен 0, если нечетное - 1. При чтении опять подсчитываем бит четности и сравниваем с сохранённым. Этого достаточно, чтобы обнаружить единичную ошибку в блоке любого размера. К сожалению, двойную (или любую четную) ошибку таким образом обнаружить не удастся.

Хорошо, усложним схему. Пусть контрольных разрядов будет не один, а, скажем, 6. Представим, что бинарный блок - это число. В дополнительных 6 разрядах можно разместить число от 0 до 63. Поэтому поделим бинарный блок (как число) на 64 и запишем в контрольные разряды остаток от деления. Нет, так нельзя, это же будут просто младшие 6 разрядов бинарного блока. Поэтому делить будем на число меньше 64, лучше, если это будет простое число (не имеющее делителей), например, 61. Теперь любая ошибка чтения будет обнаруживаться с вероятностью $(100 - 1/61)$ процентов. Так устроен популярный метод [CRC](#) (Cyclic Redundancy Check), обычно в контрольном коде 32 или 64 разряда, это позволяет находить ошибки с очень высокой вероятностью. К слову, деление достаточно дорогая операция для того, чтобы применять её при каждом чтении из памяти, CRC используют на прикладном уровне, для защиты файлов или пересылке по сети.

Но как насчет исправления ошибок? Для этого обычно используются коды Хэмминга. Коды Хэмминга также применяют к блокам данных. Рассмотрим на примере.

Пусть на входе имеется 16-битный блок данных $1001\ 0110\ 1110\ 0010_2$ (младшие разряды слева, это бинарная строка а не число).

Вставим контрольные разряды внутрь блока (раздвигая разряды) так, чтобы контрольные разряды находились в позициях, равных степеням двойки т.е. 1, 2, 4, 8, ... Заполним их изначально нулями, в первой строке номера разрядов.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
0	0	1	0	0	0	1	0	0	1	1	0	1	1	1	0	0	0	0	1	0

Всего для 16 битного блока (зелёные) потребовалось 5 дополнительных разрядов (красные).

Теперь добавим 5 строк (по числу добавочных разрядов) в которые напомним в бинарном виде номера столбцов, нули для наглядности опущены. Первая строка по прежнему - номера разрядов.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
0	0	1	0	0	0	1	0	0	1	1	0	1	1	1	0	0	0	0	1	0
1		1		1		1		1		1		1		1		1		1		1
	1	1			1	1			1	1			1	1			1	1		
			1	1	1	1					1	1	1	1					1	1
							1	1	1	1	1	1	1	1						
															1	1	1	1	1	1

Первому дополнительному разряду соответствует первая строка, пятому - последняя, назовем ~~это~~-их масками разрядов.

Для каждого дополнительного разряда подсчитаем четность количества единиц, но только в тех разрядах, где стоят единицы в его маске. Запишем результат в дополнительные разряды. Например, для третьего дополнительного разряда с номером 4:

- в самом 4 разряде изначально стоит 0
- маска состоит из интервалов 4..7, 12..15, 20..21
- единицы есть в разряде, попадающих под маску: 7,13, 14, 15, 20
- всего 5 единиц, число нечетное, в четвертый разряд записываем единицу

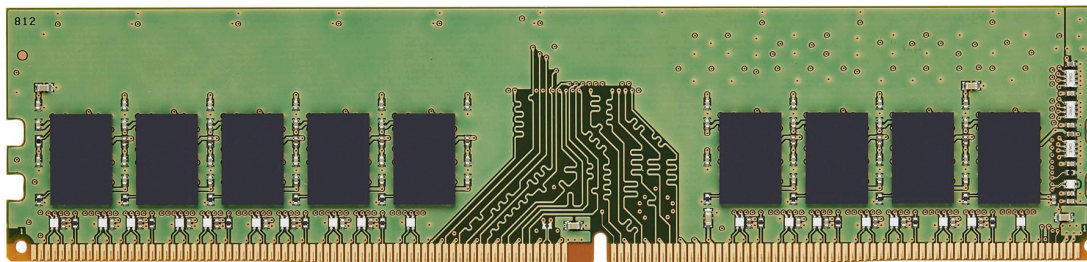
1	2	3	4	5	6	7	8	9	10	11	12	<u>13</u>	14	15	16	17	18	19	20	21
1	0	1	1	0	0	1	1	0	1	1	0	<u>1</u>	1	1	1	0	0	0	1	0
1		1		1		1		1		1		<u>1</u>		1		1		1		1
	1	1			1	1			1	1			1	1			1	1		
			1	1	1	1					1	<u>1</u>	1	1					1	1
							1	1	1	1	1	<u>1</u>	1	1						
															1	1	1	1	1	1

Кодирование завершено. Декодирование/проверка происходит совершенно также. Рассчитываются контрольные разряды и сравниваются с имеющимися. Если они совпали, значит чтение прошло корректно. Но, допустим, произошла ошибка в 13 разряде, прочитался ноль вместо единицы. Как это скажется на контрольных разрядах? Это не сложно, ошибка в 13 разряде затронет те контрольные разряды, у которых есть соответствующая единица в маске, т.е. маски 1, 3 и 4. А значит при контрольном расчете значения в этих разрядах не совпадут с прочитанными. Разряды 1, 3 и 4 в сумме дают число 13, а это именно тот разряд, в котором произошла ошибка. Исправляем 13 разряд и получаем исходное значение. Магия да и только!

Если присмотреться повнимательнее, коды Хэмминга тоже работают через остатки от деления. Сначала мы умножаем исходный блок на число специального вида, при декодировании делим на него же с проверкой остатка. Если остаток нулевой, всё хорошо, если нет - получаем номер ошибочного разряда.

Отметим, что до сих пор мы говорили про исправление одиночных ошибок. И это оправданно, возникновение ошибки - довольно редкая ситуация, двойной ошибки - весьма редкая. Тем не менее, код Хэмминга способен гарантированно обнаруживать двойные ошибки и с некоторой вероятностью тройные и т.д. Но он либо обнаруживает множественные ошибки, либо исправляет одиночные, не одновременно. Что важнее - необходимо определиться на этапе проектирования. Всё дело в восприятии ошибки. Если мы считаем, что возможны только одиночные ошибки, исправляем (но в случае двойной ошибки это лишь усугубит ситуацию). Если допускаем возникновение парных ошибок, то можем лишь зафиксировать ошибку чтения, попытаться прочесть еще раз, вдруг повезёт. Или пометить блок как сбойный и далее читать из резервного места.

В примере рассмотрено кодирование 16-разрядных блоков, на практике обычно используют 64-разрядные, для их кодирования требуется 8 дополнительных разрядов. Поэтому в планках DRAM памяти с ECC 9 микросхем вместо обычных 8.



Фиг.4.5.1 Память с ECC ([отсюда](#))

Что еще необходимо отметить. При вычислении кодов Хэмминга применяются только сдвиги разрядов и суммирование без переноса, никаких делений и умножений. Это делается с помощью относительно простой электронной схемы.